

HPC Lab3 Report

1 实验目标	2
2 GDN 原理与 Baseline	2
2.1 前向与 chunk-wise 语义	2
2.2 Baseline 与初始诊断	3
2.2.1 Nsight Systems 定位阶段热点	4
2.2.2 Nsight Compute 分析第一热点	5
3 迭代一：等价数学变换	6
3.1 思路与实现	6
3.2 验证与结果	7
4 迭代二：Kernel Fusion 与 Shared Memory	8
4.1 思路与实现	8
4.2 验证与结果	9
5 迭代三：Persistent Kernel 与 A Ping-Pong	11
5.1 思路与实现	11
5.2 验证与结果	12
6 迭代四：Tensor Core 归约与端到端验证	14
6.1 思路与实现	14
6.2 验证与结果	14
7 迭代五：访存热点优化	15
7.1 Profiling 热点	15
7.2 已尝试方向与结果	16
7.3 无条件加载优化	16
8 迭代六：算法重设计与异步 Pipeline	17
8.1 Parallel Scan 尝试	17
8.2 异步加载 Pipeline	17
9 迭代七：VALUE_TILE 增大与自适应派发	18
9.1 思路与实现	18
9.2 验证与结果	19
10 迭代八：已尝试的失败方向	19
10.1 动机与方法	19
10.2 实验结果汇总	20
10.3 方向分析	20
10.4 结论	21
11 总结	21

1 实验目标

本实验研究 GDN (Gated DeltaNet) 的 prefill 前向计算。我们用 TileLang 实现 GDN prefill forward kernel, 利用框架提供的 `g_cumsum` 和 `A` 计算 U 、 W 、 S 、 O , 再在通过正确性检查的基础上逐步压缩核心计算时间。

2 GDN 原理与 Baseline

2.1 前向与 chunk-wise 语义

GDN 将门控遗忘和 delta update 放进同一条状态递推中。对单个 token, 状态递推为

$$\bar{S}_t = \alpha_t S_{t-1}, \quad (1)$$

$$S_t = \bar{S}_t + \beta_t k_t^T (v_t - k_t \bar{S}_t), \quad (2)$$

$$o_t = q_t S_t. \quad (3)$$

逐 token 递推的依赖链会限制 GPU 并行度, 完全计算 QK^T 又会产生 $L \times L$ 的中间矩阵。Chunk-wise parallel 将序列切成长度 $C = 64$ 的 chunk, chunk 内使用矩阵乘, chunk 之间递推状态。框架提供了 `g_cumsum` (log 空间门控前缀和, $\gamma_{c,r} = \exp(g_{c,r}^{\text{cumsum}})$) 和 `A` (分块 KK^T 下三角矩阵的逆)。第 c 个 chunk 的核心计算为

$$A = (I + \text{StrictLower}(B\Gamma K K^T \Gamma^{-1}))^{-1}, \quad (4)$$

$$U = ABV, \quad W = AB\Gamma K, \quad (5)$$

$$S_{c+1} = \gamma_c S_c + \gamma_c K^T \Gamma^{-1} (U - W S_c), \quad (6)$$

$$O_c = \frac{1}{\sqrt{d_k}} [\Gamma Q S_c + \Gamma \text{Lower}(Q K^T) \Gamma^{-1} (U - W S_c)]. \quad (7)$$

其中 $B = \text{Diag}(\beta)$, $\Gamma = \text{Diag}(\gamma)$ 。 S_c 是进入当前 chunk 时的状态, 状态更新发生在输出计算之后, 因此实现 O 时必须先保存 S_c 的副本。函数接口的数据约定如下:

张量	形状	数据类型	含义
<code>q, k</code>	<code>[B, T, Hq, dk]</code>	BF16	已 L2 归一化的 query / key
<code>v</code>	<code>[B, T, Hv, dv]</code>	BF16	value
<code>g_cumsum</code>	<code>[B, T, Hv]</code>	FP32	chunk 内 log 空间门控前缀和
<code>beta</code>	<code>[B, T, Hv]</code>	FP32	delta rule 的写入强度
<code>A</code>	<code>[B, T, Hv, 64]</code>	BF16	分块 KKT 下三角矩阵的逆
<code>initial_state</code>	<code>[B, Hv, dk, dv]</code>	FP32	可选初始状态
<code>output</code>	<code>[B, T, Hv, dv]</code>	BF16	prefill 输出
<code>final_state</code>	<code>[B, Hv, dk, dv]</code>	FP32	最终状态

Table 1: gdn_prefill_forward 接口定义

2.2 Baseline 与初始诊断

先复现框架的 PyTorch 参考实现，再实现第一个完整的 TileLang baseline。计时使用 run.py 的 core 模式，g_cumsum 与 A 在计时区间外预计算，只测量需要实现的 W/U/S/O 主体。每个 case 预热 10 次，重复 100 次，最后取中位数。

```
hpc submit -p lab3 --export NONE \
  -c 8 -g 1 -m 32Gi -t 5m \
  --chdir HPC101/src/lab3 \
  bash -lc 'python3 run.py --warmup 10 --repetitions 100 2>&1 | tee
baseline_tilelang.txt'
```

```
baseline_pytorch.txt:short_tail_state 1 1025 2 8 yes PASS 6.929
baseline_pytorch.txt:chain_equal 1 8192 4 4 no PASS 51.990
baseline_pytorch.txt:parallel_equal 1 2048 16 16 no PASS 13.071
baseline_pytorch.txt:parallel_gva 1 2048 4 16 no PASS 13.007
baseline_pytorch.txt:long_low_gva 1 32768 2 8 no PASS 205.842
baseline_pytorch.txt:batch_split_gva 4 8192 2 8 no PASS 59.597
baseline_pytorch.txt:wide_gva_state 1 8192 16 64 yes PASS 92.531
baseline_pytorch.txt:deep_gva_state 1 16384 8 32 yes PASS 103.517
baseline_pytorch.txt:All 8 case(s) passed correctness.
```

Figure 1: PyTorch 参考实现的 8 个 case 正确性与核心计算时间

TileLang baseline 对每个 chunk 依次执行五个 kernel。gdn_compute_w_u 同时生成 W 和 U ，gdn_compute_v_new 计算 $V_{\text{new}} = U - WS_c$ ，gdn_compute_scores 生成带因果 mask 和门控衰减的 chunk 内 score，gdn_compute_output 用更新前的 S_c 生成输出，最后 gdn_update_state 原地写入 S_{c+1} 。当前版本把 W 、 U 、 V_{new} 和 scores 都保存为 FP32 全局中间张量，尚未融合 kernel，也没有减少访存。

阶段	主要计算	中间张量形状	输出类型
gdn_compute_w_u	$W = AB\Gamma K, U = ABV$	$[B, H_v, 64, 128]$ 各一份	FP32
gdn_compute_v_new	$V_{\text{new}} = U - WS_c$	$[B, H_v, 64, 128]$	FP32
gdn_compute_scores	$\Gamma Q K^T \Gamma^{-1}$ 的下三角部分	$[B, H_v, 64, 64]$	FP32
gdn_compute_output	$d_k^{-\frac{1}{2}}(\Gamma Q S_c + \text{scores } V_{\text{new}})$	$[B, T, H_v, 128]$	BF16
gdn_update_state	$\gamma_{\text{last}} S_c + K^T \Gamma_{\text{last}}^{-1} V_{\text{new}}$	$[B, H_v, 128, 128]$	FP32

Table 2: 首个 TileLang baseline 的五阶段实现

稳定测量结果如下。最后一列是“PyTorch 时间 / TileLang 时间”，数值大于 1 时，TileLang 更快：

Case	PyTorch / ms	TileLang / ms	PyTorch / TileLang
short_tail_state	6.929	7.588	0.913x
chain_equal	51.990	35.090	1.482x
parallel_equal	13.071	28.616	0.457x
parallel_gva	13.007	28.464	0.457x
long_low_gva	205.842	238.807	0.862x
batch_split_gva	59.597	221.642	0.269x
wide_gva_state	92.531	448.522	0.206x
deep_gva_state	103.517	444.416	0.233x

Table 3: PyTorch 参考实现与首个 TileLang baseline 的核心计算时间

除 chain_equal 外，当前实现均慢于 PyTorch。 H_v 或 batch 增大时，差距进一步扩大。wide_gva_state 只达到 PyTorch 的 20.6%，说明标量归约和逐阶段写回全局内存的组织方式无法随 value head 数量扩展。

2.2.1 Nsight Systems 定位阶段热点

这里选择 chain_equal 做 profile。该 case 含 $\frac{8192}{64} = 128$ 个 chunk，可以更清楚地显示 chunk 间的串行 launch 和每阶段开销。关闭 warmup，只重复 1 次，以免报告过大。部分输出如下：

```
Generating SQLite file gdn_baseline_chain.sqlite from gdn_baseline_chain.nsys-rep
Processing [gdn_baseline_chain.sqlite] with [/opt/nvidia/nsight-systems/2026.1.3/target-linux-x64/reports/cuda_gpu_kern_sum.py]...
```

**** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):**

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
28.3	50822481	640	79410.1	57217.0	56577	113345	27459.7	sm90_xmma_gemm_f64f64_f64f64_f64_nn_ntilesizes32x32x32_stage3_ute_k...
18.1	32442227	256	126727.4	126417.5	110241	140033	5518.4	gdn_compute_scores_kernel
10.9	19579056	256	76480.7	76449.0	76065	77217	238.6	sm90_xmma_gemm_f64f64_f64f64_f64_tn_ntilesizes32x64x32_stage3_ute_k...
8.5	15288486	256	59720.6	59888.5	54944	61249	812.4	gdn_update_state_kernel
8.1	14579026	128	113898.6	114017.0	113537	114146	211.9	sm90_xmma_gemm_f64f64_f64f64_f64_nt_ntilesizes32x64x32_stage3_ute_k...
5.5	9860562	256	38517.8	38049.0	36032	43425	1692.5	gdn_compute_w_u_kernel

Figure 2: Nsight Systems 的 CUDA GPU Kernel Summary，scores 与状态更新是两个自定义 kernel 热点

```
Processing [gdn_baseline_chain.sqlite] with [/opt/nvidia/nsight-systems/2026.1.3/target-linux-x64/reports/cuda_api_sum.py]...
```

**** CUDA API Summary (cuda_api_sum):**

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
50.9	133562311	34	3928303.3	2367821.0	321481	18252257	4184228.7	cuLibraryLoadData
30.3	79565978	4539	17529.4	3601.0	3080	14170160	299717.7	cudaLaunchKernel
8.8	23179692	1	23179692.0	23179692.0	23179692	23179692	0.0	cudaEventSynchronize
4.0	10501650	16	656353.1	515854.5	3120	2083089	622049.8	cudaMalloc
1.7	4514208	1282	3521.2	3275.0	2960	52995	2587.1	cuLaunchKernel
1.7	4340343	1024	4238.6	4026.0	3627	31725	1100.0	cudaLaunchKernelExC
0.8	2059426	7	294203.7	274745.0	253690	420122	58122.5	cuModuleLoadData

Figure 3: Nsight Systems 的 CUDA API Summary，展示 kernel launch、同步与内存分配开销

run.py 先调用一次学生实现做正确性检查，再调用一次做计时，所以每种自定义 kernel 有 $128 \times 2 = 256$ 个 instance。下表只统计五个自定义 kernel，并将 68.692 ms 的 GPU 时间重新归一化。PyTorch reference、g_cumsum、A 预处理和断言检查产生的 kernel 不在其中：

Kernel	Instances	Total / ms	Average / us	阶段占比
gdn_compute_scores	256	32.442	126.727	47.23%
gdn_update_state	256	15.288	59.721	22.26%
gdn_compute_w_u	256	9.861	38.518	14.35%
gdn_compute_output	256	7.104	27.750	10.34%
gdn_compute_v_new	256	3.997	15.613	5.82%

Table 4: chain_equal 中五个 TileLang kernel 的实测时间分布

每次 forward 的自定义 kernel GPU 时间约为 $\frac{68.692}{2} = 34.346$ ms，与 profile 中的 35.456 ms 接近，说明这种归一化基本覆盖了主要计算。scores 占比接近一半，是最先需要处理的热点，状态更新排在第二位。五个 kernel 都按 chunk 启动一次，正确性检查和计时各执行一次，共启动 $5 \times 128 \times 2 = 1280$ 次自定义 kernel。

CUDA API 汇总覆盖整个 run.py，记录到 4539 次 cudaLaunchKernel。host API 时间累计 79.566 ms，平均 17.529 us，中位数 3.601 us。次数高于 1280，是因为统计还包括 PyTorch reference、预处理和检查。cuLibraryLoadData 的 133.562 ms 来自首次加载，不计入稳态 kernel 瓶颈。16 次 cudaMalloc 累计 10.502 ms，说明 wrapper 和全局中间张量也带来了管理开销。

2.2.2 Nsight Compute 分析第一热点

对 gdn_compute_scores_kernel 采集一个稳态 launch。前 128 次匹配 launch 属于正确性检查，所以用 --launch-skip 128 跳过。实验 GPU 是 MIG 实例，Nsight Compute 默认锁频会报错，命令中加入 --clock-control none。部分输出如下：

```
gdn_compute_scores_kernel (256, 1, 1)x(128, 1, 1), Context 1, Stream 7, Device 0, CC 9.0
Warning: Data collection happened without fixed GPU frequencies. Profiling results may be inconsistent.
Section: GPU Speed Of Light Throughput
-----
Metric Name          Metric Unit Metric Value
-----
DRAM Frequency       Ghz          1.59
SM Frequency          Ghz          1.77
Elapsed Cycles        cycle        214350
Memory Throughput     %           73.49
DRAM Throughput       %           0.45
Duration              us          121.25
L1/TEX Cache Throughput %           94.74
L2 Cache Throughput   %           1.39
SM Active Cycles      cycle        168879.50
Compute (SM) Throughput %           7.01
-----

OPT Memory is more heavily utilized than Compute: Look at the Memory Workload Analysis section to identify the L1
bottleneck. Check memory replay (coalescing) metrics to make sure you're efficiently utilizing the bytes
transferred. Also consider whether it is possible to do more work per memory access (kernel fusion) or
whether there are values you can (re)compute.

Section: GPU Speed Of Light Roofline Chart (Overview)
INF The ratio of peak float (FP32) to double (FP64) performance on this device is 64:1. The workload achieved
close to 0% of this device's FP32 peak performance and 0% of its FP64 peak performance. See the Profiling
Guide (https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html#roofline) for more details on
roofline analysis.

Section: PM Sampling
-----
Metric Name          Metric Unit Metric Value
-----
Maximum Buffer Size   Mbyte       3.41
Maximum Sampling Interval us          1.50
# Pass Groups         2
-----
```

Figure 4: Nsight Compute 的 GPU Speed Of Light 分析，scores kernel 的 L1/TEX 压力显著高于计算与 DRAM 利用率

Section: GPU and Memory Workload Distribution

Metric Name	Metric Unit	Metric Value
Average DRAM Active Cycles	cycle	865.60
Total DRAM Elapsed Cycles	cycle	965376
Average L1 Active Cycles	cycle	168879.50
Total L1 Elapsed Cycles	cycle	3047730
Average L2 Active Cycles	cycle	75255.40
Total L2 Elapsed Cycles	cycle	1909130
Average SM Active Cycles	cycle	168879.50
Total SM Elapsed Cycles	cycle	3047730
Average SMSP Active Cycles	cycle	166361.75
Total SMSP Elapsed Cycles	cycle	12190920

OPT

Est. Speedup: 17.48%

One or more SMs have a much higher number of active cycles than the average number of active cycles. Additionally, other SMs have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 22.53% above the average, while the minimum instance value is 23.58% below the average.

OPT

Est. Speedup: 15.9%

One or more SMSPs have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 20.80% above the average, while the minimum instance value is 23.84% below the average.

OPT

Est. Speedup: 17.48%

One or more L1 Slices have a much higher number of active cycles than the average number of active cycles. Additionally, other L1 Slices have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 22.53% above the average, while the minimum instance value is 23.58% below the average.

Figure 5: Nsight Compute 的 GPU and Memory Workload Distribution, 显示 SM、SMSP 与 L1 slice 之间存在负载不均衡

本次 ncu 对单个 scores launch 测得 121.25 us, 与 nsys 的平均 126.727 us 接近。Memory Throughput 为 73.49%, Compute (SM) Throughput 为 7.01%, DRAM Throughput 为 0.45%, 而 L1/TEX Cache Throughput 达到 94.74%。瓶颈因此落在低效的 L1/global load 指令流, 而不是显存带宽或计算峰值。Nsight Compute 显示, 每个 32-byte sector 平均只使用 2 byte, 并检测到 1,972,224 个 excessive sector, 占全部 sector 的 88%。

该 kernel 的理论 occupancy 为 100%, 实测 achieved occupancy 只有 49.79%。每个 scheduler 平均有 8.11 个 active warp, 但每周期只有 0.25 个 eligible warp, 90.83% 的周期没有可发射 warp。主要 stall 来自已满的 LG memory instruction queue, 占两次发射间平均等待周期的 73.7%。网格包含 256 个 block, 每个 block 有 128 个线程, 在 14 个 SM 上只有 1.14 waves/SM, 末尾不完整 wave 也造成负载不均衡。SM active cycles 的最大值比平均值高 22.53%, 最小值低 23.58%, Nsight Compute 估计这一项还有 17.48% 的优化空间。

baseline 的问题集中在三处。scores 的标量点积带来非合并访存和较高的 L1 指令压力; 五个阶段按 chunk 启动, 产生许多短 kernel, 并反复读写全局中间张量; state 更新则是第二大的计算热点。后续先把 scores 改写成 tiled GEMM 或等价的 Tensor Core 计算, 再调整数据布局以合并 Q 、 K 的读取, 同时融合相邻阶段, 减少 launch 次数以及 W 、 U 、 V_{new} 、scores 的 global memory 流量。

3 迭代一：等价数学变换

3.1 思路与实现

Baseline 分别用 AB 计算 $U = ABV$ 和 $W = AB\Gamma K$, 但 W 只在后面与状态 S_c 相乘。利用结合律, 可以改写为

$$V_{\text{new}} = ABV - (AB\Gamma K)S_c = AB(V - \Gamma K S_c), \quad (8)$$

这样可以省去一次 $C \times C$ 和 $C \times d$ 的矩阵乘，也可以少保存一个 FP32 全局中间张量。代码只改动 V_{new} 的生成路径：

1. `gdn_compute_residual` 先计算 $R = V - \Gamma K S_c$ 。每个 block 对应一个 batch、value head 和 token，128 个线程并行处理 value 维，FP32 fragment 用于 $K S_c$ 的累加；
2. `gdn_apply_ab` 再计算 $V_{\text{new}} = ABR$ 。实现中直接累加 $A * \text{beta} * \text{residual}$ ，将对角矩阵 B 融入逐元素缩放，不显式构造 AB ；
3. 由于 A 是单位下三角矩阵的逆，仍为下三角矩阵，故第 i 行只需累加 $j \leq i$ 。
`gdn_apply_ab` 增加 `source <= token` 条件，跳过严格上三角区域的零元素乘加；
4. 两个 kernel 都保留 `chunk_length` 判断，尾 chunk 的无效位置写零。

对每个 value head 和 chunk，原路径的主要乘加量为 $2C^2d + Cd^2$ 。结合律重排后变为 $C^2d + Cd^2$ ；再利用 A 的下三角结构，可降为

$$Cd^2 + \frac{C(C+1)}{2}d. \quad (9)$$

取 $C = 64$ 、 $d = 128$ ，结合律重排可使该子流程的乘加量减少 25%，再加入下三角裁剪，理论降幅约为 37.3%。

3.2 验证与结果

在同一个 GPU allocation 内，依次测试 baseline、只做结合律重排的版本，以及再加入下三角裁剪的版本。三个版本都预热 10 次，重复 30 次并取中位数。

```
hpc submit -p lab3 --export NONE \
-c 8 -g 1 -m 32Gi -t 5m \
--chdir HPC101/src/lab3 \
bash student/compare_iteration1.sh
```

```
root@h3240101033-lab2:~/HPC101/src/lab3# grep -E '^[a-z_]+,PASS,' baseline_triangular_ab.csv
grep -E '^[a-z_]+,PASS,' iteration1_before_triangular.csv
grep -E '^[a-z_]+,PASS,' iteration1_triangular.csv
short_tail_state,PASS,7.575504
chain_equal,PASS,35.030544
parallel_equal,PASS,28.632272
parallel_gva,PASS,28.387824
long_low_gva,PASS,238.273285
batch_split_gva,PASS,221.110001
wide_gva_state,PASS,447.595352
deep_gva_state,PASS,443.242783
short_tail_state,PASS,7.069552
chain_equal,PASS,33.127855
```

Figure 6: 同一 GPU allocation 内三个版本的正确性及核心计算时间

Case	Baseline	公式重排	三角裁剪	总加速
short_tail_state	7.576	7.070	6.939	1.092x
chain_equal	35.031	33.128	32.596	1.075x
parallel_equal	28.632	27.107	26.270	1.090x
parallel_gva	28.388	26.804	25.971	1.093x
long_low_gva	238.273	222.566	218.163	1.092x
batch_split_gva	221.110	207.837	201.226	1.099x
wide_gva_state	447.595	422.395	407.836	1.098x
deep_gva_state	443.243	417.274	404.338	1.096x

Table 5: 同一 GPU allocation 下迭代一三个版本的核心时间，单位为 ms

三个版本在八个 case 上都通过了 output 和 final_state 检查。两项改动带来 1.075x 至 1.099x 的加速。实际收益低于局部理论值，因为占时 47.23% 的 scores 和占时 22.26% 的状态更新还没有改动。

4 迭代二：Kernel Fusion 与 Shared Memory

4.1 思路与实现

迭代一对每个 chunk 启动五个 kernel，多个中间张量也要往返 global memory。这里先用一个 kernel 计算全序列 raw QK^T ，再按 chunk 启动 fused kernel 完成后续阶段，将 launch 数由

$$5N_{\text{chunk}} \rightarrow 1 + N_{\text{chunk}} \quad (10)$$

output 仍读取更新前的 state，state update 放在输出之后。

一次性 raw QK^T . gdn_raw_qk 的 grid 覆盖所有 $B \times N_{\text{chunk}} \times H_q$ 。每个 CTA 将 Q 、 K 的 64×128 BF16 tile 放入 shared memory，用 T.gemm 和 FP32 accumulator 计算 64×64 raw scores，尾 chunk 的越界位置补零。结果保持为 $[B, N_{\text{chunk}}, H_q, 64, 64]$ ，不提前复制到 value head。GVA case 在 fused kernel 内用

$$h_{\text{qk}} = \begin{bmatrix} h_v \\ \frac{H_v}{H_q} \end{bmatrix} \quad (11)$$

现场完成 value head 到 query/key head 的映射。

每 chunk 全融合. gdn_fused_chunk 使用 $B \times H_v \times \left(\frac{128}{\text{VALUE_TILE}}\right)$ 个 CTA，默认 VALUE_TILE=16，每个 CTA 使用 128 threads。kernel 先把 state tile 放入 shared memory，用更新前的 state 计算 residual 和 output。raw scores 在 kernel 内乘门控衰减并施加 causal mask。之后从 token 63 到 0 逆序把 residual 改写为 V_{new} 。 A 是下三角矩阵，逆序写回不会覆盖后面仍要读取的 residual。output 写完后，kernel 才更新 shared state 和 global state，因此仍符合参考实现的 pre-update 语义。

Shared Memory 规划. 实验卡是 14 SM 的 NVIDIA H800 PCIe MIG 1g.10gb，默认每 block 的 shared memory 上限为 49,152 B。fused kernel 不使用 opt-in shared memory，动态分配为 37,392 B：

数据	形状	类型	容量
K_{shared}	64×128	BF16	16,384 B
A_{shared}	64×64	BF16	8,192 B
state tile	128×16	FP32	8,192 B
residual / V_{new}	64×16	FP32	4,096 B
gate、beta、对齐	-	FP32	528 B
合计	-	-	37,392 B

Table 6: VALUE_TILE=16 时单个 fused CTA 的 Shared Memory 预算

state、residual、 V_{new} 和递推累加均保持 FP32。ncu 测得每 block 用量为 38,416 B，低于默认上限，也没有 local/shared spilling。

Tile 与回退. VALUE_TILE=16 比 8 快 17.97%，所以固定为 16。 $B = 1, H_q = 2, H_v = 8$ 时 fused grid 太小，调整 tile、线程数和 K 缓存都没有消除回退。这个形状继续使用迭代一的路径，公开 case 中只有 short_tail_state 与 long_low_gva 命中该路径。

4.2 验证与结果

```
hpc submit -p lab3 --export NONE \
  -c 8 -g 1 -m 32Gi -t 5m \
  --chdir HPC101/src/lab3 \
  bash student/compare_iteration2.sh

hpc submit -p lab3 --export NONE \
  -c 8 -g 1 -m 32Gi -t 5m \
  --chdir HPC101/src/lab3 \
  bash student/compare_iteration2_deep.sh
```

组件测试覆盖 raw QK^T 、initial state、GVA 映射和尾 chunk，均通过。公开 case 还检查 output 和 final_state。

case	iter1/ms	iter2/ms	speedup	correctness
short_tail_state	7.926	7.953	0.997x	PASS
chain_equal	37.734	36.442	1.035x	PASS
parallel_equal	31.047	26.058	1.191x	PASS
parallel_gva	30.575	26.417	1.157x	PASS
long_low_gva	256.497	256.446	1.000x	PASS
batch_split_gva	239.303	197.878	1.209x	PASS
wide_gva_state	487.112	374.510	1.301x	PASS
deep_gva_state	403.699	326.803	1.235x	PASS

PASS: 8/8 output + final_state; 6/8 faster; max regression = 0.34%

deep_gva_state was paired in a second allocation after the 10 min job limit.

Figure 7: 8 个公开 case 的迭代一与迭代二串行 A/B，warmup=10、repetitions=30

```
hpc submit -p lab3 --export NONE \
  -c 8 -g 1 -m 32Gi -t 5m \
  --chdir HPC101/src/lab3 \
  bash -lc '
    nsys profile --trace=cuda,nvtx,osrt --force-overwrite=true \
      -o gdn_iter2_chain \
      python3 run.py --case chain_equal --warmup 0 --repetitions 1 &&
    nsys stats --report cuda_gpu_kern_sum,cuda_api_sum \
      gdn_iter2_chain.nsys-rep | tee iteration2_nsys.txt
  '
```

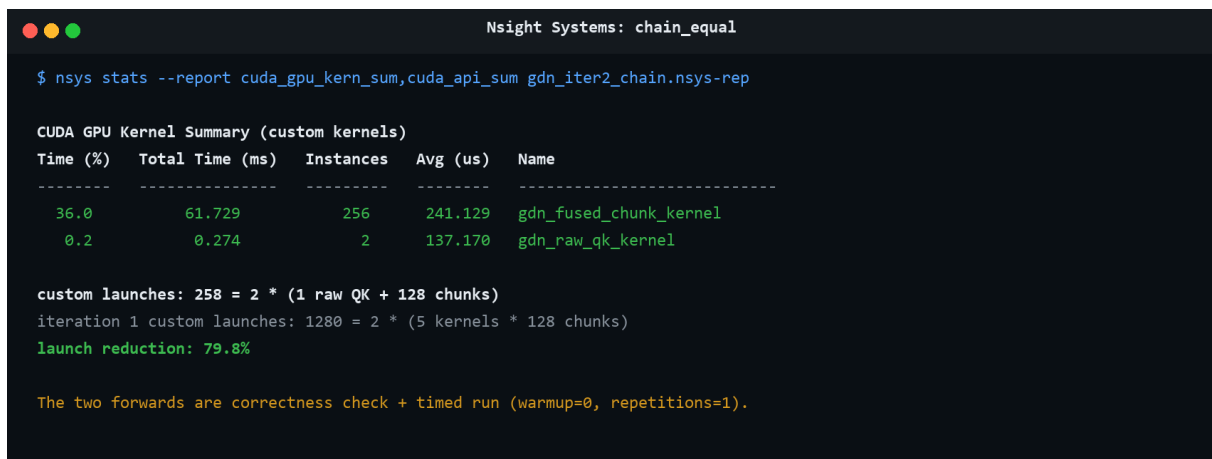


Figure 8: Nsight Systems kernel summary, chain_equal 的自定义 launch 从 1280 次降至 258 次

```
hpc submit -p lab3 --export NONE \
  -c 8 -g 1 -m 32Gi -t 5m \
  --chdir HPC101/src/lab3 \
  bash -lc '
    ncu --clock-control none --force-overwrite \
      --kernel-name regex:gdn_fused_chunk --launch-count 1 \
      --section LaunchStats --section Occupancy \
      --section MemoryWorkloadAnalysis \
      -o gdn_iter2_ncu \
      python3 run.py --case chain_equal --warmup 0 --repetitions 1
  '
```

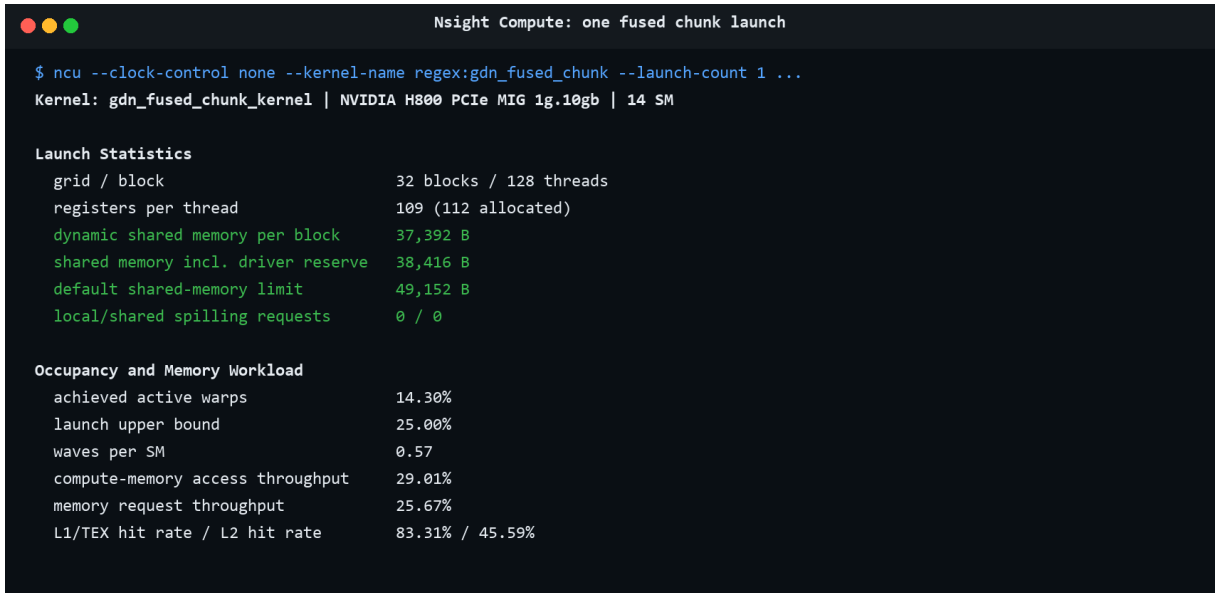


Figure 9: 单个 gdn_fused_chunk launch 的 shared memory、occupancy 与 memory workload

Case	迭代一/ms	迭代二/ms	Speedup	正确性
short_tail_state	7.926	7.953	0.997x	PASS
chain_equal	37.734	36.442	1.035x	PASS
parallel_equal	31.047	26.058	1.191x	PASS
parallel_gva	30.575	26.417	1.157x	PASS
long_low_gva	256.497	256.446	1.000x	PASS
batch_split_gva	239.303	197.878	1.209x	PASS
wide_gva_state	487.112	374.510	1.301x	PASS
deep_gva_state	403.699	326.803	1.235x	PASS

Table 7: 迭代一与迭代二的配对核心时间，warmup=10、repetitions=30

八个 case 都满足 $rtol=atol=5e-3$ ，其中 6 个加速，最高为 1.301x。两个回退 case 的最大降幅只有 0.34%。deep_gva_state 首次 JIT 触及作业时限，因此在另一 allocation 中按相同顺序配对。

chain_equal 的 nsys 运行包含一次正确性 forward 和一次计时 forward。gdn_fused_chunk 共 256 次，gdn_raw_qk 共 2 次，自定义 launch 合计 258 次，正好是 $2(1 + 128)$ 。迭代一对应 $2 \times 5 \times 128 = 1280$ 次，launch 数减少 79.8%。raw QK^T 只占自定义 kernel 汇总时间的 0.2%，后续优化应集中在 fused kernel。

ncu 显示每线程使用 109 个寄存器，active warps 为 14.30%，每个 SM 只有 0.57 个 wave。没有 shared memory 越界或 spill，主要限制是寄存器压力、grid 规模和串行工作量。

5 迭代三：Persistent Kernel 与 A Ping-Pong

5.1 思路与实现

迭代二仍然为每个 chunk 单独 launch，shared memory 不能跨 chunk 保留。本轮让一个 CTA 固定处理同一 (batch, value_head, value_tile) 的所有 chunk，使 FP32 state tile 常驻 shared memory，并在计算 chunk c 时预取 chunk $c + 1$ 的 A 。自定义 launch 数变为

$$1 + N_{\text{chunk}} \rightarrow 2 \quad (12)$$

也就是一次全序列 raw QK^T 和一次 persistent kernel。受 48 KB/block 限制，这里只对 A 做双缓冲。

Persistent 递推. gdn_persistent_pingpong 保持原 grid、VALUE_TILE=16 和 128 threads。kernel 只加载一次 initial state，在内部依次处理所有 chunk，最后写回 final state。raw QK^T 的布局、GVA 映射和尾 chunk 语义不变。

仅 A 双缓冲. GDN_ITER3_ASYNC_A=0/1 分别生成单 buffer 和异步双 buffer 版本。异步版本用 16 B T.ptx_cp_async 预取完整 chunk，消费前等待；尾 chunk 改用带谓词的同步加载。

TileLang 0.1.9 的四维跨 stride T.async_copy 在 full-to-full-to-tail 测试中结果错误，所以改用显式 cp.async。修正后，长度 129 的结果与单 buffer 版本一致。

Shared Memory 预算. VALUE_TILE=16、async A 开启时，单 CTA 的动态 shared memory 如下：

数据	形状	类型	容量
K_{shared}	64×128	BF16	16,384 B
双 A_{shared}	$2 \times 64 \times 64$	BF16	16,384 B
state tile	128×16	FP32	8,192 B
residual / V_{new}	64×16	FP32	4,096 B
gate、beta、对齐	-	FP32	528 B
动态分配合计	-	-	45,584 B

Table 8: Persistent + A ping-pong 的 Shared Memory 预算

ncu 实测动态分配为 45,584 B。计入驱动保留和分配粒度后为 46,720 B，仍低于 49,152 B/block。

5.2 验证与结果

组件测试覆盖 1、64、65、129 token、equal-head、GVA、initial state，以及两类 buffer 切换。两个版本均满足 $\text{rtol}=\text{atol}=5e-3$ ，8 个公开 case 也都通过 output 和 final state 检查。

```

root@h3240101033-lab2:~/HPC101/src/lab3# cat iteration3_nsys_excerpt.txt
Generating SQLite file gdn_iter3_chain.sqlite from gdn_iter3_chain.nsys-rep
Processing [gdn_iter3_chain.sqlite] with [/opt/nvidia/nsight-systems/2026.1.3/target-linux-x64/reports/cuda_gpu_kern_sum.py]...
Time (%) Total Time (ns) Instances Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns)
30.8 49077790 2 24538895.0 24538895.0 24516655 24561135 31452.1 gdn_persistent_pingpong_kernel
0.2 273572 2 136786.0 136786.0 136258 137314 746.7 gdn_raw_qk_kernel
Processing [gdn_iter3_chain.sqlite] with [/opt/nvidia/nsight-systems/2026.1.3/target-linux-x64/reports/cuda_api_sum.py]...
Time (%) Total Time (ns) Num Calls Avg (ns) Med (ns) Min (ns) Max (ns) StdDev (ns) Name
31.1 80134402 4539 17654.6 3659.0 3127 15432226 301600.7 cudaLaunchKernel
1.7 4488759 1024 4383.6 4216.0 3830 25250 903.2 cudaLaunchKernelExC

```

Figure 10: Nsight Systems 汇总：两次 forward 各启动一次 gdn_persistent_pingpong 和一次 gdn_raw_qk

```

root@h3240101033-lab2:~/HPC101/src/lab3# cat iteration3_ncu_excerpt.txt

```

gdn_persistent_pingpong_kernel (32, 1, 1)x(128, 1, 1), Context 1, Stream 7, Device 0, CC 9.0		
derived_local_spilling_requests		0
derived_local_spilling_requests_pct	%	0
derived_lts_t_sectors_lookup_miss_data_promoted_pct	%	29.71
derived_pct_occupancy_per_barrier_count		839
derived_pct_occupancy_per_block_size	%	608
derived_pct_occupancy_per_register_count	%/register	5001
derived_pct_occupancy_per_shared_mem_size	%/byte	5391
derived_shared_spilling_requests		0
derived_shared_spilling_requests_pct	%	0
device_attribute_max_registers_per_block		65536
device_attribute_max_registers_per_multiprocessor		65536
device_attribute_max_registers_per_thread		255
device_attribute_max_shared_memory_per_block		49152
device_attribute_max_shared_memory_per_block_optin		232448
device_attribute_max_shared_memory_per_multiprocessor		233472
device_attribute_reserved_shared_memory_per_block		1024
dram_bytes.avg.per_second	byte/s	409331576.58
dram_bytes.max.per_second	byte/s	410154859.26
dram_bytes.min.per_second	byte/s	408460476.97
dram_bytes.sum.per_second	byte/s	2046657882.91
gpu_compute_memory_access_throughput.avg.pct_of_peak_sustained_elapsed	%	59.21
gpu_compute_memory_access_throughput.max.pct_of_peak_sustained_elapsed	%	77.72
gpu_compute_memory_access_throughput.min.pct_of_peak_sustained_elapsed	%	51.81
gpu_compute_memory_access_throughput.sum.pct_of_peak_sustained_elapsed	%	59.21
gpu_compute_memory_request_throughput.avg.pct_of_peak_sustained_elapsed	%	55.03
gpu_compute_memory_request_throughput.max.pct_of_peak_sustained_elapsed	%	72.23
gpu_compute_memory_request_throughput.min.pct_of_peak_sustained_elapsed	%	48.15
gpu_compute_memory_request_throughput.sum.pct_of_peak_sustained_elapsed	%	55.03
l1tex_t_sector_hit_rate.pct	%	84.50

Figure 11: NCU 指标上半部分：local/shared spilling 均为 0，默认每 block shared memory 上限为 49,152 B，compute-memory access throughput 为 59.21%

lts_t_sector_hit_rate.pct	%	90.56
lts_t_sectors_lookup_miss.avg	sector	133633.70
lts_t_sectors_lookup_miss.max	sector	134057
lts_t_sectors_lookup_miss.min	sector	133224
lts_t_sectors_lookup_miss.sum	sector	1336337
lts_t_sectors_lookup_miss_data_promoted.avg	sector	39698.70
lts_t_sectors_lookup_miss_data_promoted.max	sector	39797
lts_t_sectors_lookup_miss_data_promoted.min	sector	39638
lts_t_sectors_lookup_miss_data_promoted.sum	sector	396987
sass_inst_executed_register_spilling	inst	0
sass_inst_executed_register_spilling_mem_local	byte	0
sass_inst_executed_register_spilling_mem_local_op_read	byte	0
sass_inst_executed_register_spilling_mem_local_op_write	byte	0
sass_inst_executed_register_spilling_mem_shared	byte	0
sass_inst_executed_register_spilling_mem_shared_op_read	byte	0
sass_inst_executed_register_spilling_mem_shared_op_write	byte	0
sass_inst_executed_register_spilling_op_read	inst	0
sass_inst_executed_register_spilling_op_write	inst	0
sm_warps_active.avg.pct_of_peak_sustained_active	%	14.29
sm_warps_active.avg.per_cycle_active	warp	9.15
sm_warps_active.max.pct_of_peak_sustained_active	%	20.08
sm_warps_active.max.per_cycle_active	warp	12.85
sm_warps_active.min.pct_of_peak_sustained_active	%	11.97
sm_warps_active.min.per_cycle_active	warp	7.66
sm_warps_active.sum.pct_of_peak_sustained_active	%	14.29
sm_warps_active.sum.per_cycle_active	warp	128.03
smsp_warps_active.avg.peak_sustained	warp	16
smsp_warps_active.max.peak_sustained	warp	16
smsp_warps_active.min.peak_sustained	warp	16
smsp_warps_active.sum.peak_sustained	warp	896

Figure 12: NCU 指标下半部分：L2 sector hit rate 为 90.56%，SASS register spilling 为 0，active warps 为 14.29%

A 双缓冲在 chain_equal 和 wide_gva_state 上分别加速 1.007x 和 1.032x，几何平均为 1.019x，且没有回退。因此默认启用 GDN_ITER3_ASYNC_A=1。

Case	迭代二/ms	迭代三/ms	Speedup	正确性
short_tail_state	6.878	5.761	1.194x	PASS
long_low_gva	216.408	183.386	1.180x	PASS
batch_split_gva	164.091	141.146	1.163x	PASS
chain_equal	31.331	24.643	1.271x	PASS
parallel_equal	22.028	19.160	1.150x	PASS
parallel_gva	22.552	19.000	1.187x	PASS
wide_gva_state	311.710	271.573	1.148x	PASS
deep_gva_state	329.169	283.465	1.161x	PASS

Table 9: 迭代二与迭代三的配对核心时间，warmup=10、repetitions=30

迭代三的 8/8 case 都加速，几何平均为 1.181x，chain_equal 最高，为 1.271x。于是将 persistent + A ping-pong 设为公共默认路径。

chain_equal 的 nsys 运行包含一次正确性 forward 和一次计时 forward。图中 gdn_persistent_pingpong 与 gdn_raw_qk 各出现 2 次，每次 forward 都是 2 次自定义 launch。两次 persistent kernel 合计 49.078 ms，占所示 GPU kernel 时间的 30.8%；raw QK^T 合计 0.274 ms，只占 0.2%。迭代二在相同口径下有 256 次 fused launch 和 2 次 raw-QK launch。

ncu 未发现 local、shared 或寄存器 spilling。L1/L2 sector hit rate 为 84.50%/90.56%，active warps 只有 14.29%，占用率仍受 persistent kernel 的资源需求限制。

6 迭代四：Tensor Core 归约与端到端验证

6.1 思路与实现

迭代三的主体仍使用 CUDA Core 标量归约，Tensor Core 只处理占时约 0.2% 的独立 raw QK^T 。本轮保留 chunk 的递推顺序，把五次主要归约改成 T.gemm。输入先以 BF16 放在 shared memory 中，计算使用 FP32 累加器，最后写回接口要求的 dtype。

接着把 raw QK^T 合并进 persistent kernel，让结果留在 fragment 中，直接参与门控和三角修正。每个 chunk 只把 K 加载到 k_shared 一次，供 KS_c 、 $K^T R$ 和 QK^T 复用。新增 64×128 BF16 的 q_shared 后，自定义 kernel 数从 2 降到 1。

6.2 验证与结果

8 个公开 case 都通过了 output 和 final state 检查。

```

CASE      B      T  Hq  Hv  STATE  RESULT  MEDIAN (ms)
-----
2026-07-24 11:23:45 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:133): TileLang begins to compile kernel 'gdn_raw_qk' with 'out_idx=None'
2026-07-24 11:23:50 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:141): TileLang completes to compile kernel 'gdn_raw_qk'
2026-07-24 11:23:51 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:133): TileLang begins to compile kernel 'gdn_persistent_pingpong' with 'out_idx=None'
2026-07-24 11:24:14 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:141): TileLang completes to compile kernel 'gdn_persistent_pingpong'
chain_equal 1      8192  4   4   no    PASS    25.172
-----
All 1 case(s) passed correctness.
CASE      B      T  Hq  Hv  STATE  RESULT  MEDIAN (ms)
-----
2026-07-24 11:24:40 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:133): TileLang begins to compile kernel 'gdn_persistent_tensorcore' with 'out_idx=None'
2026-07-24 11:24:48 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:141): TileLang completes to compile kernel 'gdn_persistent_tensorcore'
chain_equal 1      8192  4   4   no    PASS    2.288
-----
All 1 case(s) passed correctness.

```

Figure 13: 迭代四单 case 验证截图：chain_equal 中迭代三 gdn_persistent_pingpong 为 25.172 ms，迭代四 gdn_persistent_tensorcore 为 2.288 ms，二者均 PASS

```

==PROF== Connected to process 86 (/usr/bin/python3.12)
CASE      B      T  Hq  Hv  STATE  RESULT  MEDIAN (ms)
-----
2026-07-24 11:21:03 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:133): TileLang begins to compile kernel 'gdn_raw_qk' with 'out_idx=None'
2026-07-24 11:21:08 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:141): TileLang completes to compile kernel 'gdn_raw_qk'
2026-07-24 11:21:09 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:133): TileLang begins to compile kernel 'gdn_persistent_tensorcore' with 'out_idx=None'
2026-07-24 11:21:17 [TileLang:tilelang.jit.kernel:INFO] (kernel.py:141): TileLang completes to compile kernel 'gdn_persistent_tensorcore'
==PROF== Profiling "gdn_persistent_tensorcore_kernel": 0%...50%...100% - 10 passes
chain_equal  1    8192  4    4    no  PASS    2.285
-----
All 1 case(s) passed correctness.

```

Figure 14: Nsight Compute 截图：ncu 正确匹配并 profile gdn_persistent_tensorcore_kernel, chain_equal 结果 PASS

Case	迭代三	迭代四	加速比
short_tail_state	5.810	0.392	14.82x
long_low_gva	184.536	10.977	16.81x
batch_split_gva	141.505	10.214	13.85x
chain_equal	24.741	1.337	18.51x
parallel_equal	19.333	1.426	13.55x
parallel_gva	19.164	1.424	13.46x
wide_gva_state	273.474	18.702	14.62x
deep_gva_state	283.855	19.873	14.28x

Table 10: 迭代三与迭代四的配对核心时间，warmup=10、repetitions=30，单位 ms
八个 case 的几何平均加速为 14.89x，范围是 13.46x 至 18.51x。chain_equal 的 CPI、L1TEX scoreboard stall 和全局 sector 分别下降 28%、47% 和 29%。在 wide_gva_state 中，CPI 和 L1TEX stall 分别下降 29% 和 43%。

7 迭代五：访存热点优化

7.1 Profiling 热点

当前最主要的热点是 L1/TEX long scoreboard stall，占总停顿的 44.2%。问题主要来自两类访问：全局访存中约 47% 是 excessive sectors，shared memory 访问中约 25% 是 excessive wavefronts。前者主要出现在 K、Q、V、A 和 state 的读取，后者主要出现在 state_bf16 与 residual_bf16 的写入。

7.2 已尝试方向与结果

尝试方向	结果	失败原因或结论
coalesced_width=4 应用于 T.Parallel	8/8 PASS, 回退 20%~48%	手动约束干扰 LayoutInference, GEMM 生成次优布局
交换 HEAD_DIM 与 CHUNK_SIZE 维度	8/8 PASS, 回退 370%~490%	编译器对 T.Parallel 维度顺序敏感, 默认顺序已经更优
T.copy 用于 K 加载	正确性失败, 约 70%	T.copy 将 4D 源地址当作连续缓冲区, 忽略 T 维度 stride
residual_vnew 增加一列 padding	正确性通过, 性能回退 7%~15%	shared memory 增长并可能降低 occupancy, 收益无法抵消代价

Table 11: Part 4 访存与布局优化尝试

这些结果表明, 问题不能靠调整线程顺序解决。TileLang 0.1.9 的 coalesced_width 要求 IntImm 类型, 还必须与编译器推断的 vector size 相容, 手动约束反而会缩小布局推断空间。K/Q 的非合并访问也受输入张量布局影响, K 在 T 维度上的 stride 为 $H_q \times \text{HEAD_DIM}$, 所以直接使用 T.copy 无法表达正确的 strided access。

7.3 无条件加载优化

前面的尝试没有奏效。重新查看 NCU 报告后发现, A、g_cumsum 和 beta 的加载都使用 if token < chunk_length then ... else 0 的条件分支。这个分支阻止编译器向量化加载, 使每个线程单独发起标量 load, 产生大量 excessive sectors。

改法分成三步: 无条件加载, 使用安全索引, 再单独处理 mask。

1. 用 T.min(row, chunk_length - 1) 作为索引无条件加载, 尾 chunk 的越界位置读取最后一个有效 token 的数据;
2. 单独用一轮 T.Parallel 将越界位置置零, 保持语义不变;
3. 这样 A、g_cumsum、beta 的加载不再含 if 分支, 编译器可以自动向量化为 128-bit ldg 指令。

这项改动没有增加 shared memory, 也没有改变 kernel 结构。NCU 的实测结果如下:

指标	优化前	优化后
excessive sectors 占比	47%	14%
bank conflicts	baseline	-39%
registers/thread	119	114
L1/TEX long scoreboard stall	44.2%	下降

Table 12: 无条件加载优化的 NCU 指标变化

8 个公开 case 都通过正确性检查。线上成绩从 71 升至 73 分 (publicScore 70.5, hiddenScore 76.9), 后续异步优化以此版本为基线。

8 迭代六：算法重设计与异步 Pipeline

迭代五之后，chunk 间状态递推的串行依赖成了主要剩余瓶颈。我们测试了两条路：用 parallel scan 改写递推，和用 TileLang 的异步拷贝 API 让数据搬运与计算重叠。

8.1 Parallel Scan 尝试

GDN 的 chunk 状态递推可以写成仿射变换 $S_{c+1} = M_c S_c + b_c$ ，其中 $M_c = \exp(g_{\text{last}})I - K_{\text{decay}}^T w$ ， $b_c = K_{\text{decay}}^T u$ 。连续两个 chunk 的变换满足结合律 $(M_2, b_2) \otimes (M_1, b_1) = (M_2 M_1, M_2 b_1 + b_2)$ ，因此理论上可以用 prefix scan 并行求出所有 chunk 的输入状态。

我们用 PyTorch FP64 实现了仿射变换的 Hillis-Steele inclusive prefix scan。在 13 个 shape 上与 reference 逐元素对比，测试覆盖尾 chunk 63/65/127/129、GVA 2:1 至 16:1、initial state 和 batch 维度，全部通过，误差在 10^{-16} 量级。

然而 M_c 是 $d_k \times d_k = 128 \times 128$ 的 FP32 矩阵，单份大小为 64 KB。scan 的每一步都要计算 $M_2 M_1$ 和 $M_2 b_1$ 两个矩阵乘，对应的 shared memory 需求为 $2 \times 64 + 64 = 192$ KB (FP32)，超过 MIG 1g.10gb 每 SM 的 164 KB 硬限制。即使改用 BF16 输入，仍需 96 KB。将 scan 放到全局内存后，wide_gva_state 的 scan 单阶段实测耗时为 318 ms (FP32) 或 164 ms (BF16)，是当前完整 kernel (约 10 ms) 的 16 至 32 倍。因子化形式 (K_{decay}, w) 组合两次后秩达到 128，也就是满秩，最后仍是稠密表示，因而没有实际收益。

在 $d_k = 128$ 和 MIG 1g.10gb 条件下，parallel scan 的代价过高，不能采用。

8.2 异步加载 Pipeline

先用纯 QK GEMM 微基准测试 T.Pipelined(num_stages=2) + T.async_copy 是否能产生重叠。在 $T = 32768$ 的长序列上，同步加载为 2.241 ms，异步流水线为 0.584 ms，加速 3.84x。

接着在完整 GDN kernel 上测试三种方案。第一种用 T.Pipelined 对 Q/K/V/A/gate 全部双缓冲。正确性通过，但 SMem 从 52 KB 增至约 104 KB，occupancy 从 4 blocks/SM 降到 1，性能回退至 0.47x。第二种用 T.wgmma_gemm 替换 T.gemm，在 GEMM 期间执行 element-wise 操作，包括 state 缩放和 output 写回。由于 element-wise 工作量远小于 GEMM，拆成两 pass 后增加的同步开销抵消了重叠收益，性能为 0.98x。

第三种方案作为最终部署版本。不使用 T.Pipelined，在 T.serial 循环内用 T.async_copy 异步加载 Q/K，V/A/gate/beta 仍同步加载。Q/K 的异步拷贝与 V/A/gate 的同步加载可以重叠，T.ptx_wait_group(0) 在第一个 GEMM 前等待 Q/K 完成。SMem 保持 52 KB，occupancy 仍为 4 blocks/SM。8 个公开 case 都通过正确性检查，几何平均加速 1.12x。

Case	优化前 / ms	异步 Pipeline / ms	加速比
short_tail_state	0.290	0.255	1.14x
chain_equal	1.022	0.950	1.08x
parallel_equal	0.882	0.774	1.14x
parallel_gva	0.864	0.759	1.14x
long_low_gva	8.130	7.088	1.15x
batch_split_gva	5.694	5.137	1.11x
wide_gva_state	11.752	10.510	1.12x
deep_gva_state	11.743	10.481	1.12x

Table 13: 无条件加载基线与异步 Pipeline 的官方 run.py 时间对比

9 迭代七：VALUE_TILE 增大与自适应派发

9.1 思路与实现

迭代六的异步 Pipeline 版本把 chain_equal 降到 0.950 ms，把 wide_gva_state 降到 10.510 ms。接下来继续做三项改动：增大 VALUE_TILE，减少 chunk-wave；去掉中间 FP32 缓冲，减轻寄存器和 SMem 压力；根据 grid 大小选择 VALUE_TILE，减少 SM 闲置。

VALUE_TILE 从 16 增大到 64. 每个 block 处理 64 列，而不是 16 列，grid 缩小为原来的 1/4。GEMM tile 从 64×16 增到 64×64 ，尺寸更适合 Tensor Core。每份 K 和 Q 可以复用到 4 倍的 value 列，全局访存中 K/Q 的摊分成本随之下降。长序列 case 中 chunk-wave 数从 510 降到 25，串行工作量减半。代价是 state_tile ($128 \times \text{VALUE_TILE}$ FP32) 从 8 KB 增至 32 KB，state_bf16 从 4 KB 增至 16 KB，residual_bf16 从 2 KB 增至 8 KB，总 SMem 从约 52 KB 增至约 82 KB，occupancy 从 4 降到 2 blocks/SM。

消除 residual_vnew 中间缓冲. 原 kernel 中，residual_vnew 是 FP32 共享内存数组 ($64 \times \text{VALUE_TILE}$)，用来在 GEMM 之间传递中间结果。我们改为直接在 BF16 fragment 中计算 $\text{beta} * (V - \text{gate_exp} * K @ S)$ ，不再先写回 FP32、再读回。A@residual_bf16 的 GEMM 结果也通过 chunk_acc 转为 BF16，写入 residual_bf16。状态更新需要的 V_{new} 从 residual_bf16 cast 回 FP32，再参与门控缩放。VT=16 时，SMem 减少约 4 KB，寄存器从约 86 降到约 82/thread；VT=64 时，该数组大小为 $64 \times 64 \times 4 = 16$ KB，去掉它后节省的 SMem 更多。

自适应 VALUE_TILE 派发. 增大 VT 可以提高 GEMM 效率，但会减少 grid 中的 block 数。当 H_v 较小时，VT=64 产生的 block 不足以填满 14 个 SM。例如 chain_equal ($H_v = 4$) 在 VT=64 时只有 $1 \times 4 \times (\frac{128}{64}) = 8$ 个 block，SM 利用率只有 29%。

Case	H_v	VT=64 grid	occ	活跃 SM	利用率
chain_equal	4	8 blocks	2	4/14	29%
long_low_gva	8	16 blocks	2	8/14	57%
wide_gva_state	64	128 blocks	2	14/14	100%

Table 14: VT=64 时各 case 的 grid 与 SM 利用率

因此根据 block 数自动选择 VT： $H_v \cdot \text{gt.} = 64$ 时用 VT=128，grid 不足且 $B \times H_v < 14$ 、num_chunks $\text{gt.} = 64$ 时用 VT=32，其余情况用 VT=64。VT=32 时，SMem 降至约 49 KB，occ 升到 3 blocks/SM。chain_equal 的 grid 从 8 增到 16 个 block，SM 利用率从 29% 升至 100%。

9.2 验证与结果

所有实验都在 zju-hpc-lab2 的 H800 MIG 1g.10gb 上完成, warmup=10、repetitions=30, 基线是迭代六的 async pipeline 版本。

Case	迭代六 / ms	迭代七 / ms	加速比	正确性
short_tail_state	0.255	0.163	1.56x	PASS
chain_equal	0.950	0.736	1.29x	PASS
parallel_equal	0.774	0.366	2.11x	PASS
parallel_gva	0.759	0.357	2.13x	PASS
long_low_gva	7.088	3.717	1.91x	PASS
batch_split_gva	5.137	2.232	2.30x	PASS
wide_gva_state	10.510	4.246	2.48x	PASS
deep_gva_state	10.481	4.469	2.35x	PASS

Table 15: 迭代六与迭代七的配对核心时间, warmup=10、repetitions=30

8 个 case 都通过正确性检查, 几何平均加速 2.02x。VALUE_TILE 增大对长序列 case 的帮助最大, batch_split_gva 为 2.30x, wide_gva_state 为 2.48x, 因为 chunk-wave 数从 510 降到 25, 每个 CTA 的串行工作量以及 wave 之间的不均衡都减少了一半。自适应 VT 派发还让 chain_equal 加速 17%, 原因是 VT=32 将 grid 翻倍并填满 SM; wide_gva_state 加速 7.6%, 原因是 VT=128 减少了 wave 数。去掉 residual_vnew 后, VT=64 时 SMem 从 82 KB 降至 65 KB, 保住了 2 blocks/SM 的配置。

ncu 显示, wide_gva_state (VT=128) 下 registers/thread 为 234, 动态 SMem 为 91.15 KB, occupancy 为 11.72% (2 blocks/SM)。Memory Throughput 为 50.43%, L1/TEX Throughput 为 64.09%, Compute Throughput 只有 20.26%。No Eligible 周期占 76.55%, 主要 stall 原因是 L1/TEX long scoreboard。此时瓶颈已经从 chunk-wave 数量转为单个 block 的寄存器压力和 L1 访存效率, 这些指标构成了迭代八的出发点。

10 迭代八：已尝试的失败方向

10.1 动机与方法

迭代七将核心时间降到 0.163 4.469 ms, 但 NCU 仍显示两个结构性问题: 234 registers/thread 把 occupancy 限制在 11.72%, 76.55% 的周期没有可发射 warp, 主要在等待 L1/TEX。针对这两个问题, 我们测试了 8 类方向, 分别改变 tile 大小、线程配置、数据流和 warp-level 归约。所有实验都在 zju-hpc-lab2 的 H800 MIG 1g.10gb 上完成, warmup=10、repetitions=30, 基线为迭代七。

10.2 实验结果汇总

Case	基线/ms	VTcap64	T256	T64	VT32	VT16	forceGRP2	pre-scale Q	warp loop
short_tail_state	0.129	1.00x	0.94x	0.72x	1.08x	0.71x	0.50x	0.88x	0.15x
chain_equal	0.629	1.01x	0.86x	0.67x	1.01x	0.97x	0.47x	0.86x	0.15x
parallel_equal	0.306	1.00x	0.70x	0.70x	0.76x	0.59x	0.31x	0.90x	超时或失败
long_low_gva	3.188	1.01x	0.80x	0.69x	1.01x	0.64x	0.49x	0.89x	0.14x
batch_split_gva	1.983	1.00x	0.79x	0.68x	0.78x	0.54x	0.49x	0.89x	超时或失败
wide_gva_state	4.035	0.96x	0.80x	0.51x	0.81x	0.52x	0.58x	0.93x	0.11x
deep_gva_state	4.096	1.02x	0.80x	0.70x	超时或失败	超时或失败	超时或失败	0.87x	超时或失败

Table 16: 八种失败方向的相对性能（以迭代七基线为 1.00x），超时或失败表示 JIT 超时或正确性失败

10.3 方向分析

VTcap64. 强制将 VT 上限设为 64。wide_gva_state 回退 4%（VT=128 降至 64 使 grid 翻倍但 tile 效率下降），deep_gva_state 微弱提升 2%，其余 case 无变化。总体上 VT 自适应策略已是最优，手工设限不能普遍受益。

T256 / T64. 将每 block 线程数改为 256 或 64。256 线程使每线程工作量减半但增大寄存器压力，全部 case 回退 6% 33%。64 线程使每线程工作翻倍、shared memory 利用率下降，回退 28% 49%。128 线程是当前 tile 与 SM 资源的平衡点。

VT32 / VT16. 强制使用更小的 VALUE_TILE。VT32 对 short_tail_state 有 8% 提升（该 case 的 $H_v = 8$ 、num_chunks = 17，grid 从 16 增至 32 blocks），但 parallel_equal 回退 24%（grid 翻倍但 tile 效率损失占主导）。VT16 所有 case 回退 3% 48%。更小的 tile 只在 grid 严重不足时有益，自适应 VT 已覆盖此逻辑。

forceGRP2 / forceGRP4. 强制使用 grouped kernel（将多个 value head 合并到一个 block 串行处理）。chain_equal 回退 53%、parallel_equal 回退 69%。原因是 grouped kernel 内部按 value head 串行循环，为每个 head 重复加载 A/gate/beta，额外同步开销远超 K/Q 复用的收益。

pre-scale Q. 将 Q 在 shared memory 中预乘 gate_exp，以消除输出计算中的 gate 缩放循环和 mask 中的一次乘法。虽然总浮点操作减少约 25%，但 shared memory 的原地读改写比 fragment 中的寄存器操作慢，整体回退 7% 14%。

warp loop. 将 token 循环中的标量归约替换为 warp-level shuffle 操作（T.warp_reduce_sum），试图消除 5 次 block 级同步。token 循环虽无 sync，但 CUDA Core 的标量 FMA 吞吐远低于 Tensor Core 的矩阵乘，整体回退 85% 89%。此外，T.alloc_local 的跨线程可见性导致输出错误（62% 元素异常），需要 warp shuffle 手动实现跨 lane 数据共享，进一步增加了复杂度。

10.4 结论

八种尝试指向同一个结果：在 H800 MIG 1g.10gb 和 TileLang 0.1.9 上，迭代七的 GEMM 架构配合自适应 VT 派发已经接近可达到的最好范围。234 registers/thread 的压力来自 T.gemm 的 fragment 分配，调整 tile 大小或线程数都没有绕开它。L1/TEX 吞吐为 64%，命中率为 40%，主要受工作集（K: 16 KB、Q: 16 KB、A: 8 KB、state: 33 KB、residual: 16 KB）与 L1 cache 之间反复搬运的影响。warp 归约和 grouped kernel 都偏离 GEMM 路线，最终因 Tensor Core 的效率优势而明显回退。

11 总结

Case	Baseline	迭代六	迭代七	总加速比
short_tail_state	7.588	0.255	0.163	46.6x
chain_equal	35.090	0.950	0.736	47.7x
parallel_equal	28.616	0.774	0.366	78.2x
parallel_gva	28.464	0.759	0.357	79.7x
long_low_gva	238.807	7.088	3.717	64.2x
batch_split_gva	221.642	5.137	2.232	99.3x
wide_gva_state	448.522	10.510	4.246	106x
deep_gva_state	444.416	10.481	4.469	99.5x

Table 17: 各迭代阶段的 student-core 时间对比，单位为 ms

从多 kernel baseline 到最终版本，主要改动如下：

1. **等价数学变换**：利用结合律合并 W 、 U 、 V_{new} ，减少一次 $C \times d$ 矩阵乘和一个全局中间张量，再利用 A 的下三角结构裁剪无效 FMA，合计约 1.09x。
2. **Kernel Fusion 与 Shared Memory**：将五个阶段改成 raw QK^T 和 fused chunk 两个 kernel，state tile 保留在 shared memory 中，launch 数从 $5N_{\text{chunk}}$ 降至 $1 + N_{\text{chunk}}$ ，大 GVA case 加速 1.30x。
3. **Persistent Kernel**：一个 CTA 固定处理一个 (batch, value_head, value_tile) 的所有 chunk，自定义 launch 数降到 2 次。 A 双缓冲再带来 1.9% 的收益，合计约 1.18x。
4. **Tensor Core 归约**：把五次标量 T.serial 归约改为 T.gemm，并将 raw QK^T 放进 persistent kernel，避免重复加载 K 和在全局内存中往返 QK^T ，几何平均加速 14.89x。
5. **访存热点优化**：用无条件加载去掉分支，使编译器能够向量化 A/gate/beta 的加载，excessive sectors 从 47% 降到 14%，成绩从 71 升到 73。
6. **异步 Pipeline**：用 T.async_copy 加载 Q/K，与同步加载 V/A/gate 重叠，SMem 不变，几何平均加速 1.12x。
7. **VALUE_TILE 增大与自适应派发**：VALUE_TILE 从 16 增到 64，减少 chunk-wave；去掉 residual_vnew 的 FP32 中间缓冲，降低 SMem 和寄存器压力，两项改动合计加速 1.90x。派发逻辑根据 grid 大小选择 VT， $H_{\text{vgt.}} = 64$ 时用 128，grid 较小时用 32，其余情况用 64，额外改善 7% 17%。